

Bid Auction Protocol

BAP / 1.0 — Application-Layer Protocol Specification

Project 1: Socket Programming

Distributed Bid Auction System (Client – Proxy – Server, over TCP)

Pomdet Foong

Kasetsart University — Computer Science

1. Objective

This program is a **distributed bid auction system**. It lets multiple clients place monetary bids on numbered auction items (item IDs 1–200) over a network, and guarantees that at any point in time there is exactly one recognized highest bid (and highest bidder) per item. The system is used the way a live auction house's back-end would be used: a client submits a bid; the system either accepts it as the new leading bid or rejects it because a higher bid already exists; and the current leaderboard for every item is always available.

The purpose of building it this way — rather than as a single monolithic server — is to demonstrate a horizontally **sharded** client-server architecture: item IDs are partitioned into ranges, each range is owned end-to-end by one proxy/server pair, and a directory service (the **registry**) lets clients and proxies find each other dynamically instead of relying on a fixed, compiled-in address book.

1.1 Application characteristics

- **Architecture:** multi-tier client–server, not peer-to-peer. Four distinct roles: Client, Proxy, Server, Registry.
- **Sharding:** the item ID space is partitioned into disjoint ranges (e.g. 1–100, 101–200). Each range is served by exactly one Proxy–Server pair.
- **Statefulness:** Proxies are stateless routers/validators. Servers are stateful — each Server is the sole authority for the current highest bid on every item in its range.
- **Dynamic discovery:** Proxies register their range and address with the Registry at startup; nothing is hardcoded between processes. Clients query the Registry per bid to learn which Proxy to contact.
- **Request–response, not streaming:** every exchange is one request followed by exactly one response, then the logical operation is complete (the underlying TCP connection may be reused or re-opened per request).
- **Persistence:** each Server appends every bid decision to an audit log and rewrites a snapshot file on disk, so its state survives a process restart.

1.2 Transport layer: TCP, and why

This application uses **TCP** (via Go's net package, `net.Listen("tcp", ...) / net.Dial("tcp", ...)`), not UDP. The justification:

- **Reliability is required, not optional.** A bid is a semantically significant, one-shot transaction. If a bid message were silently dropped (as UDP allows), the client would believe it had bid when the server never saw it — an unacceptable correctness gap for anything resembling money changing hands.
- **Ordering matters within a shard.** Whether Bob's \$150 bid is processed before or after Carol's \$120 bid determines the outcome of the auction for that item. TCP's in-order delivery on a single connection preserves this; UDP gives no ordering guarantee at all.
- **Connection-oriented semantics fit the protocol's error model.** BAP/1.0 needs to distinguish "the peer is unreachable" from "the peer replied with an error status." TCP's connection setup (three-way handshake) surfaces unreachability immediately as a connection error, which the Proxy uses to return a 503 Service Unavailable to the client without ambiguity.

- **Message sizes are small and traffic is low-frequency** (one JSON object per bid), so TCP's slightly higher per-connection overhead versus UDP is not a meaningful cost here — the correctness guarantees are worth far more than the latency UDP might save.

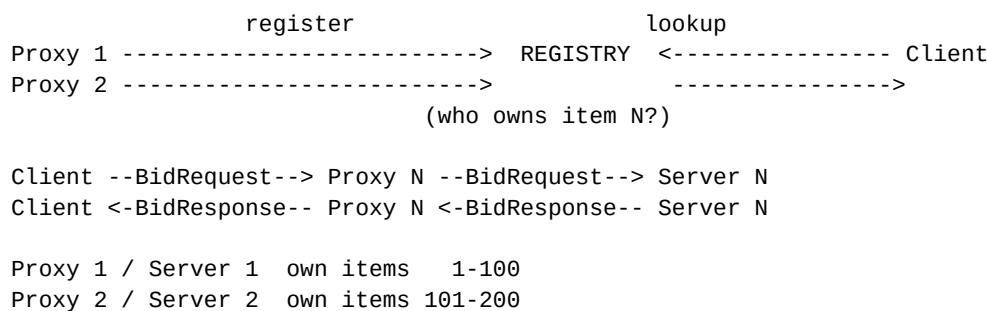
2. Protocol Name and Framing

The application-layer protocol designed for this system is named the **Bid Auction Protocol**, abbreviated **BAP**, current version **1.0** — identified on the wire as the string "BAP/1.0", carried in a `protocol` field on every message, the same way an HTTP request line carries "HTTP/1.1".

Wire format: every BAP/1.0 message is a single JSON object, encoded as UTF-8 and terminated by a newline character (`\n`). Since JSON payloads for this protocol never themselves contain a literal newline, splitting the TCP byte stream on `\n` is sufficient to recover message boundaries. This is a request–response protocol: for every message sent by a Client or Proxy, exactly one response is read back on the same connection before the next message may be sent.

BAP/1.0 defines two independent message families that share the same framing and status-code style but serve different purposes: the **Bid message family** (Client–Proxy–Server) and the **Registry message family** (Proxy/Client–Registry).

2.1 Architecture overview



3. Bid Message Family (Client ↔ Proxy ↔ Server)

3.1 BidRequest

Sent by the Client to a Proxy, and forwarded unchanged by that Proxy to its paired Server.

Field	Type	Description
<code>protocol</code>	string	Protocol identifier, always "BAP/1.0".
<code>client_name</code>	string	The bidder's display name.
<code>item_id</code>	int	The auction item being bid on (1–200).
<code>bid_amount</code>	float	The bid amount in the auction's currency unit.

Example on the wire:

```
{"protocol":"BAP/1.0","client_name":"Alice","item_id":42,"bid_amount":100}
```

3.2 BidResponse

Sent by the Server back to the Proxy, and relayed unchanged by that Proxy to the Client.

Field	Type	Description
protocol	string	Protocol identifier, always "BAP/1.0".
status_code	int	Numeric result code (see §3.3).
status_phrase	string	Human-readable phrase paired with the code.
reason	string	(optional) Extra detail, e.g. why a bid was rejected.
item_id	int	(optional) The item this response concerns.
highest_bid	float	(optional) The current highest bid on that item.
highest_bidder	string	(optional) Name of the current highest bidder.

Example on the wire (accepted):

```
{"protocol": "BAP/1.0", "status_code": 200, "status_phrase": "OK - Bid Accepted",
  "item_id": 42, "highest_bid": 100, "highest_bidder": "Alice"}
```

Example on the wire (rejected):

```
{"protocol": "BAP/1.0", "status_code": 409, "status_phrase": "Conflict - Bid Too Low",
  "reason": "bid too low", "item_id": 42, "highest_bid": 150, "highest_bidder": "Bob"}
```

3.3 Bid status codes

Code	Phrase	Meaning
200	OK - Bid Accepted	This bid is now the highest bid on the item.
409	Conflict - Bid Too Low	A higher bid already exists for this item.
404	Not Found - Item Out Of Range	The contacted Proxy does not own this item's ID range.
503	Service Unavailable - Server Unreachable	The Proxy's paired Server could not be reached.

4. Registry Message Family (Proxy/Client ↔ Registry)

The Registry is what makes Proxies **dynamic**: instead of a shared, compiled-in address table, each Proxy announces itself to the Registry on startup, and each Client asks the Registry, per bid, which Proxy currently owns the relevant item ID. A single message type, `RegistryMessage`, is used for both operations, distinguished by its `type` field.

4.1 RegistryMessage (request)

Field	Type	Description
protocol	string	Protocol identifier, always "BAP/1.0".
type	string	"register" (sent by a Proxy) or "lookup" (sent by a Client).
proxy_id	int	(register) Unique ID of the registering Proxy.
low / high	int	(register) Inclusive item ID range this Proxy owns.
proxy_host / proxy_port	string	(register) Address the Proxy listens on.
item_id	int	(lookup) The item ID the Client wants to bid on.

Example — a Proxy registering:

```
{"protocol": "BAP/1.0", "type": "register", "proxy_id": 1, "low": 1, "high": 100,
```

```
"proxy_host": "127.0.0.1", "proxy_port": "7001"}
```

Example — a Client looking up item 42:

```
{"protocol": "BAP/1.0", "type": "lookup", "item_id": 42}
```

4.2 RegistryResponse

Field	Type	Description
protocol	string	Protocol identifier, always "BAP/1.0".
status_code	int	Numeric result code (see §4.3).
status_phrase	string	Human-readable phrase paired with the code.
reason	string	(optional) Extra detail, e.g. why a lookup failed.
proxy_id, proxy_host, proxy_port	various	(lookup only) Address of the Proxy that owns the requested item.

Example — successful lookup:

```
{"protocol": "BAP/1.0", "status_code": 200, "status_phrase": "OK",  
  "proxy_id": 1, "proxy_host": "127.0.0.1", "proxy_port": "7001"}
```

4.3 Registry status codes

Code	Phrase	Meaning
200	OK	Registration accepted, or lookup found a matching Proxy.
404	Not Found - No Proxy Registered For Item	No registered Proxy currently owns this item ID.
400	Bad Request - Unknown Message Type	The "type" field was neither "register" nor "lookup".

5. Example Message Flow: Two Bids on the Same Item

The following trace shows the full BAP/1.0 exchange for two clients bidding on item 42, which belongs to Proxy 1 / Server 1. Alice bids first and is accepted; Bob then outbids her and is also accepted; Carol then tries to underbid and is rejected.

- Alice's client -> Registry: lookup item_id=42
Registry -> Alice's client: 200 OK (proxy_id=1, 127.0.0.1:7001)
- Alice's client -> Proxy 1: BidRequest{client_name=Alice, item_id=42, bid_amount=100}
Proxy 1 -> Server 1: BidRequest{client_name=Alice, item_id=42, bid_amount=100}
Server 1 -> Proxy 1: 200 OK - Bid Accepted (highest_bid=100, highest_bidder=Alice)
Proxy 1 -> Alice's client: 200 OK - Bid Accepted (highest_bid=100, highest_bidder=Alice)
- Bob's client bids 150 on item 42 (same lookup+bid sequence as above) -> 200 OK - Bid Accepted
- Carol's client -> Proxy 1: BidRequest{client_name=Carol, item_id=42, bid_amount=120}
Proxy 1 -> Server 1: BidRequest{client_name=Carol, item_id=42, bid_amount=120}
Server 1 -> Proxy 1: 409 Conflict - Bid Too Low (highest_bid=150, highest_bidder=Bob)
Proxy 1 -> Carol's client: 409 Conflict - Bid Too Low (highest_bid=150, highest_bidder=Bob)

Design notes

- A Proxy never decides accept/reject itself — that decision requires the current highest bid, which only the owning Server holds. The Proxy's only independent decision is a 404 (item out of its range) or a 503 (its Server is unreachable).

- The Registry is consulted once per bid attempt, not once per connection, since the Registry's answer for a given item does not change unless a Proxy re-registers.
- Every hop prints the literal JSON it sends and receives (see accompanying source code), so a live capture in Wireshark or a terminal transcript can be matched line-for-line against this specification.